

Threads Locks

CS311 Operating Systems, Fall 2020

Taj Muhammad Khan

Introduction

- Threads are created by the programmer
- BUT Threads are **scheduled** by the OS!
- Imagine multiple threads updating shared data:

balance = balance + 1;

Problem

- Critical section being entered by multiple threads simultaneously
- Critical section being interrupted in the middle
- Only if we could make the critical section mutually exclusive!
 - i.e. only one thread can enter a critical section at any given instant
- This would have the effect of the critical section being atomic!
- Locks help us achieve this.

Locks

- Locks are constructs provided by the OS (or libraries)
- Locks can be acquired and released
- With OS (and hardware) support, it is ensured that only **one** thread can acquire a given lock at any given time!
- We use locks to protect our critical sections in multiple threads:

```
1  lock_t mutex; // some globally-allocated lock 'mutex'
2  ...
3  lock(&mutex);
4  balance = balance + 1;
5  unlock(&mutex);
```

Locks ... contd.

- A lock is a variable which can be in any of two states:
 - Available (or free, or unlocked)
 - Acquired (or held, or locked)
- When a thread calls `lock()` on a particular lock variable, it:
 - if the lock is in free state, it acquires the lock, and `lock()` function returns
 - If the lock is in held state, it will block and the `lock()` function will not return until it has acquired the lock
- A thread can free an acquired lock by calling `unlock()` on it.

Locks ... contd.2

- Locks provide a minimal amount of control over scheduling to the programmer.
- fine-grained locking vs. coarse-grained locking

Pthread locks (mutexes)

```
1  pthread_mutex_t lock = PTHREAD_MUTEX_INITIALIZER;  
2  
3  Pthread_mutex_lock(&lock); // wrapper; exits on failure  
4  balance = balance + 1;  
5  Pthread_mutex_unlock(&lock);
```

Locks – implementation

- Goals:
 - Correctness - Achieve mutual exclusion
 - Fairness - Be fair
 - Performance - Be efficient
 - Single thread single processor
 - Multiple threads single processor
 - Multiple threads multiple processors

Lock – implementation ... contd.

- First solution

- Disable interrupts
- CPU has special insns for this

```
1  void lock() {  
2      DisableInterrupts();  
3  }  
4  void unlock() {  
5      EnableInterrupts();  
6  }
```

- Works for single processor systems
- Turning off interrupts is a privileged operation
- If interrupts are off, useful interrupts can be lost

Locks – implementation ... contd.2

- Second solution

- Use a variable (flag) to communicate b/w threads

```
1  typedef struct __lock_t { int flag; } lock_t;
2
3  void init(lock_t *mutex) {
4      // 0 -> lock is available, 1 -> held
5      mutex->flag = 0;
6  }
7
8  void lock(lock_t *mutex) {
9      while (mutex->flag == 1)    // TEST the flag
10         ; // spin-wait (do nothing)
11     mutex->flag = 1;           // now SET it!
12 }
13
14 void unlock(lock_t *mutex) {
15     mutex->flag = 0;
16 }
```

Locks – implementation ... contd.3

- Second solution (flag) – problems:
 - Performance (spin waiting)
 - Correctness?

Thread 1	Thread 2
call lock () while (flag == 1) interrupt: switch to Thread 2	call lock () while (flag == 1) flag = 1; interrupt: switch to Thread 1
flag = 1; // set flag to 1 (too!)	

Figure 28.2: **Trace: No Mutual Exclusion**

Locks – implementation ... contd.4

- Third solution – Use hardware support!

- test-and-set (atomic exchange) instruction

```
1  int TestAndSet(int *old_ptr, int new) {  
2      int old = *old_ptr; // fetch old value at old_ptr  
3      *old_ptr = new;      // store 'new' into old_ptr  
4      return old;          // return the old value  
5  }
```

- performed atomically

Locks – implementation ... contd.5

- test-and-set spin lock

```
1  typedef struct __lock_t {
2      int flag;
3  } lock_t;
4
5  void init(lock_t *lock) {
6      // 0: lock is available, 1: lock is held
7      lock->flag = 0;
8  }
9
10 void lock(lock_t *lock) {
11     while (TestAndSet(&lock->flag, 1) == 1)
12         ; // spin-wait (do nothing)
13 }
14
15 void unlock(lock_t *lock) {
16     lock->flag = 0;
17 }
```

- Correctness?

- Fairness?

- Performance?

Locks – implementation ... contd.6

- Use hardware support – compare-and-swap instruction

```
1  int CompareAndSwap(int *ptr, int expected, int new) {  
2      int actual = *ptr;  
3      if (actual == expected)  
4          *ptr = new;  
5      return actual;  
6  }
```

```
1  void lock(lock_t *lock) {  
2      while (CompareAndSwap(&lock->flag, 0, 1) == 1)  
3          ; // spin  
4  }
```

- Spin-lock implementation use same as test-and-set

Locks – implementation ... contd.7

- Use hardware support – load-linked and store-conditional insns
- Both instructions operate in tandem

```
1  int LoadLinked(int *ptr) {
2      return *ptr;
3  }
4
5  int StoreConditional(int *ptr, int value) {
6      if (no update to *ptr since LoadLinked to this address) {
7          *ptr = value;
8          return 1; // success!
9      } else {
10         return 0; // failed to update
11     }
12 }
```

Locks – implementation ... contd.8

- Store-conditional succeeds **only** if no intervening store has happened to that address since the last load-linked!!

```
1  void lock(lock_t *lock) {
2      while (1) {
3          while (LoadLinked(&lock->flag) == 1)
4              ; // spin until it's zero
5          if (StoreConditional(&lock->flag, 1) == 1)
6              return; // if set-it-to-1 was a success: all done
7                      // otherwise: try it all over again
8      }
9  }
10
11 void unlock(lock_t *lock) {
12     lock->flag = 0;
13 }
```

Locks – implementation ... contd.9

- Hardware support – fetch-and-add instruction

```
1  int FetchAndAdd(int *ptr) {  
2      int old = *ptr;  
3      *ptr = old + 1;  
4      return old;  
5  }
```

Locks – implementation ... contd.10

- Ticket lock

```
1  typedef struct __lock_t {
2      int ticket;
3      int turn;
4  } lock_t;
5
6  void lock_init(lock_t *lock) {
7      lock->ticket = 0;
8      lock->turn   = 0;
9  }
10
11 void lock(lock_t *lock) {
12     int myturn = FetchAndAdd(&lock->ticket);
13     while (lock->turn != myturn)
14         ; // spin
15 }
16
17 void unlock(lock_t *lock) {
18     lock->turn = lock->turn + 1;
19 }
```

Figure 28.7: Ticket Locks

Locks – implementation ... contd.11

- Spinning needlessly wastes CPU cycles!
- How do we make a more efficient approach?
- Requires OS support!

Locks – implementation ... contd.12

- Yield() system call

```
1 void init() {  
2     flag = 0;  
3 }
```

- Threads don't spin unnecessarily. Saves CPU time.

```
4  
5 void lock() {  
6     while (TestAndSet(&flag, 1) == 1)  
7         yield(); // give up the CPU  
8 }
```

```
9  
10 void unlock() {  
11     flag = 0;  
12 }
```

Locks – implementation ... contd.13

- Use Queues

```
1  typedef struct __lock_t {
2      int flag;
3      int guard;
4      queue_t *q;
5  } lock_t;
6
7  void lock_init(lock_t *m) {
8      m->flag = 0;
9      m->guard = 0;
10     queue_init(m->q);
11 }
```

```
13 void lock(lock_t *m) {
14     while (TestAndSet(&m->guard, 1) == 1)
15         ; //acquire guard lock by spinning
16     if (m->flag == 0) {
17         m->flag = 1; // lock is acquired
18         m->guard = 0;
19     } else {
20         queue_add(m->q, gettid());
21         m->guard = 0;
22         park();
23     }
24 }
25
26 void unlock(lock_t *m) {
27     while (TestAndSet(&m->guard, 1) == 1)
28         ; //acquire guard lock by spinning
29     if (queue_empty(m->q))
30         m->flag = 0; // let go of lock; no one wants it
31     else
32         unpark(queue_remove(m->q)); // hold lock
33                                     // (for next thread!)
34     m->guard = 0;
35 }
```

- Sleep instead of spinning, if lock is held.
- Park() and unpark() support provided by Solaris
- *guard* is a spin lock around *flag* and *wait queues*
- some spinning is done but only for the time we access the flag&queue
- Flag is not set to 0 on unpark()!

Futexes (let the students do this on their own)

```
1 void mutex_lock (int *mutex) {
2     int v;
3     /* Bit 31 was clear, we got the mutex (the fastpath) */
4     if (atomic_bit_test_set (mutex, 31) == 0)
5         return;
6     atomic_increment (mutex);
7     while (1) {
8         if (atomic_bit_test_set (mutex, 31) == 0) {
9             atomic_decrement (mutex);
10            return;
11        }
12        /* We have to waitFirst make sure the futex value
13         we are monitoring is truly negative (locked). */
14        v = *mutex;
15        if (v >= 0)
16            continue;
17        futex_wait (mutex, v);
18    }
19 }
```

Futexes

- MSB is the status
- All other bits keep count

```
21 void mutex_unlock (int *mutex) {  
22     /* Adding 0x80000000 to counter results in 0 if and  
23        only if there are not other interested threads */  
24     if (atomic_add_zero (mutex, 0x80000000))  
25         return;  
26  
27     /* There are other threads waiting for this mutex,  
28        wake one of them up.  */  
29     futex_wake (mutex);  
30 }
```

Two-phased locks

- If the lock is held, spin for a while to see if it's acquirable in near future.
- If not, then go to sleep.